Better Exploration of Region-Level Value Locality with Integrated Computation Reuse and Value Prediction

Youfeng Wu Dong-Yuan Chen Jesse Fang
Microprocessor Research Labs (MRL)
Intel Corporation
Santa Clara, CA 95052-8119
http://www.intel.com/research/mrl

Abstract

Computation-reuse and value-prediction are two recent techniques for improving microprocessor performance by exploiting value localities. They both aim at breaking the data dependence limit in traditional processors. In this paper, we propose a speculative multithreading scheme in which the same hardware can be efficiently used for both computation reuse and value prediction. For the SpecInt95 benchmarks, our experiment shows that the integrated approach significantly out-performs either computation reuse or value prediction alone. For example, the integrated approach improves over computation reuse from a speedup of 1.25 to 1.40, and improves over value prediction from 1.28 to 1.40. In particular, the integrated approach out-performs a computation reuse configuration that has twice as much reuse buffer entries (from a speedup 1.33 to 1.40). Furthermore, unlike the computation reuse approach, the performance of the integrated approach does not rely on value profile during region formation and thus our approach is more suitable for production systems.

1. Introduction

Recent studies [14,15,16,21,24] have demonstrated that value locality frequently presents in real-world programs. For examples, a *computation* (performed by an instruction or a region of code) may produce the same result repeatedly [22,25,5]. A load operation may fetch the recurring value from memory [15]. A store instruction may frequently write the same value to a memory location [14]. The value locality can be exploited to by-pass the redundant operations. This is especially beneficial when the operations that are on the critical path can be skipped.

Two popular techniques, namely value prediction [7,8,9,15,16,18,19,29,30] and computation reuse [5,6,22,25], have been proposed to exploit value locality in programs. Both techniques aim at reducing the execution time of programs by breaking the data dependence limit.

Value prediction is a speculative technique that uses the result values from previous execution instances of an instruction to predict the result value of the current execution instance without actually executing the instruction. The predicted value can then be used speculatively as an input to dependent instructions so that they can execute earlier than they could have, reducing the execution cycles required. If the prediction turns out to be incorrect, recovery mechanism must be employed to squash speculative results and re-execute all instructions that use the incorrectly predicted value. Various hardware-based value predictors [9,15,16,18,29,30] have been proposed, including last value predictor, stride predictor, context-based predictor, and hybrid predictor, and have been shown to improve the performance of superscalar processors. A couple of compiler-directed value prediction approaches [8,19] rely on the compiler to identify value prediction candidates and to schedule the instructions accordingly to reduce the length of critical paths.

Computation reuse [5,6,22,25], on the other hand, is a non-speculative technique that caches the inputs and results of previous execution instances of a computation. When the same computation is encountered later with previously seen inputs, the cached results are retrieved and used directly without actually executing the computation. Performance of programs improves due to skipping the execution of reusable computation and the earlier availability of the results from reusable computation, resulting in a reduction of critical path length. Both hardware-based scheme [22,25] and compiler-directed scheme [5,6] has been proposed for computation reuse.

1063-6897/01 $10.00 © 2001 IEEE

The majority of value prediction techniques focus on instruction-level prediction [8,9,15,16,18,19,29,30]. That is, they make prediction on the result of a single instruction execution one at a time. The block-level prediction technique in [7] predicts the outcomes of multiple branch instructions at the same cycle. However, the block-level prediction scheme applies only to branch instructions for predicting the direction of multiple branches. The trace-based value prediction in [18] uses value prediction to predict the outputs for a trace based on its inputs in an attempt to obtain better speculative thread-level parallelism. However, the trace is limited to a loop trace. On the other hand, several computation reuse techniques support reuse beyond the instruction level. The S_{n+d} instruction reuse scheme in [22] dynamically establishes dependence chains among reusable instructions, allowing a sequence of instructions to be reused in one cycle. The value reuse technique in [10] exploits the natural boundary in programs and treats a basic block as a unit for reuse (a basic block usually consists of 4 to 5 instructions). This approach is subsequently enhanced to divide large basic blocks into sub-blocks [11] to increase reuse opportunity and reduce hardware cost. The compiler-directed computation reuse scheme in [5] exploits reuse opportunity at the region-level. The computation regions are identified by the compiler and could include loops or complicated control flow. On the average, about 15 instructions can be skipped when a computation region is reused.

Although value prediction and computation reuse are two different techniques in exploiting value locality in programs, they can complement each other and potentially achieve better performance in an integrated approach than each of them alone. The result of an instruction may be reusable but may not be predictable. This could happen when the output values exhibit a complex pattern or no pattern at all that couldn't be captured by the value predictor. Similarly, a value may be predictable but not reusable. This could happen when the cycle of a repeating value pattern is larger than the size of the hardware reuse buffer. For example, a long sequence of constant stride values resulting from an instruction cannot be reused as the number of different values easily overflows the hardware reuse buffer.

In this paper, we proposed an integrated value prediction and computation reuse scheme that exploits value locality of programs in the region level. The compiler statically identifies general regions of code (called computation regions in this paper) that are suitable for reuse or value prediction. At runtime, the hardware reuses or predicts the outputs of a region as a whole. Our integrated value prediction and computation reuse scheme resembles the compiler-directed computation reuse scheme in [5] in the way it exploits computation reuse opportunity. But if the computation region cannot be reused, our scheme attempts to predict the results of the region and use the predicted values to advance the execution of dependent instance speculatively. We employ the speculative multithreading hardware to support value prediction and recovery when the prediction may be incorrect.

There have been extensive researches on speculative multithreading architectures [1,13,18,26] that exploit thread-level parallelism by running speculative threads concurrently with the main thread on multiple processors or processing units. Speculative threads are spawned before the necessity to execute these threads can be ascertained (i.e., control speculation) and even before the inputs to these threads are known (i.e., data speculation). To support the execution of multiple speculative threads, the processor (or system) generally employs special microarchitecture design that contains multiple execution cores[1] interconnected through special control and communication mechanisms. For example, the Multiscalar [26] and Chip-Multiprocessor [13] architectures spawn speculative threads per the annotations in the executables and concurrently runs these speculative threads on multiple processors that are interconnected into a special topology. Special hardware mechanisms are provided to communicate register values and resolve memory references among processors. The Dynamic Multithreading Processor [1] automatically detects and spawns threads speculatively at the loop and procedure boundaries and further uses value prediction to allow speculative threads to run ahead before the inputs to the threads are known. In a processor with simultaneous multithreading (SMT) pipeline, instructions from multiple threads compete for the available execution resources [22,28]. Some of the threads running on a SMT processor can be speculative [17].

Recently several speculative multithreading architectures have been proposed that contain execution cores with dedicated functionality. The DIVA architecture [2,4] contains a speculative DIVA core and a DIVA checker. The DIVA core is allowed to run ahead in program execution with aggressive speculation techniques such as branch prediction or value prediction. The DIVA checker receives the instructions executed in the DIVA core and verifies the correctness of the DIVA core execution before committing the result to architecture states. Although the DIVA architecture is designed with fault tolerance in mind, the dynamic

[1] An execution core could be a processor or a specially designed processing unit.

verification architecture can be leveraged to increase processor performance. The slipstream processor [20,27] uses a two-way multiprocessor design to improve single-program performance by running two processes created from the same program simultaneously. One of the processor runs the advanced stream (or A-stream) that may speculatively bypass ineffectual instructions and communicates its results to the other processor that runs the redundant stream (or R-stream). The R-stream executes the instructions in full and uses the outcomes from the A-stream as predictions to speed up overall program execution. The VLIW architecture proposed in [19] supports value prediction through a two-core design – the VLIW engine and the Compensation Code engine. The VLIW engine runs the original programs and may speculatively run ahead using value prediction. The Compensation Code engine is dedicated to buffer speculatively executed instructions and re-executed these instructions once a value mis-prediction is detected later.

Our proposed speculative multithreading architecture also consists of two execution cores with dedicated functionality to support integrated region-level computation reuse and value prediction. The processor supports special buffers that remember dynamically the previous execution *instances* for the designated computation regions. The instance includes information on the input registers to the computation region and their values at the time of entering the computation region and the set of output registers and their values when execution exits the computation region. When a designated computation region is encountered again in subsequent program execution, the current input registers to the region and their values are used to lookup the cached instances for that region. If an instance with matching inputs is found, the computation region is skipped and the cached outputs of the matching instance is used directly in subsequent execution. However if no instance matches the inputs, our scheme tries to utilize the information captured in the instances associated with the computation region to predict the outputs of the region. The predicted outputs of the computation region are then used speculatively to continue the execution of code after the region on the main pipeline (called a *main core*). Meanwhile, the computation region is forwarded to a checker pipeline (called a *check core*) for execution as a separate thread and its outputs from the check core are compared with the predicted outputs. If the output matches, the results from the speculative execution on the main core are committed. Otherwise, the speculative execution on the main core is discarded and execution is restarted from the correct exit of the mispredicted computation region, using the correct outputs from the check core.

For the SpecInt95 benchmarks, our experiment shows that the integrated approach out-performs either computation reuse or value prediction alone. More interestingly, the integrated approach with a hardware buffer of four instances per region out-performs computation reuse with a hardware buffer of eight instances per region. Furthermore, computation reuse relies on accurate value profile to form reusable regions and deliver performance. With the integrated approach, the performance is less sensitive to the accuracy of value profile used to select computation regions at compile time.

The rest of the paper is organized as follows. We first introduce computation reuse in Section 2. In Section 3, we discuss value prediction and the integrated hardware. In Section 4, we outline the architectural extension to support the integrated approach. In Section 5, we describe the speculative multi-threaded hardware. In Section 6, we present our experimental results, and in Section 7 we conclude the paper and provide future research directions.

2. Computation reuse and speculative reuse

The computation reuse scheme as proposed in [5] is compiler-driven and non-speculative. It uses a computation region buffer (CRB) as shown in Figure 1 to record instances for computation regions. CRB contains one entry for each computation region recently executed. Each CRB entry has space for storing a fixed number of instances for the computation region. Each instance contains a set of input registers and their values together with a set of output registers and their values. Other fields, such as the valid flag and replacement information are omitted in the figure. For simplicity, we assume that all the instances for a computation region have the same set of input registers and the same set of output registers. This may also allow the register numbers to be stored with the CRB entry rather than being repeated in the instances as shown in Figure 1.

As an example, we look at the CRB entry for region rgn_i in Figure 1. The hardware remembers instances I_1, I_2, ..., I_i for the region. Each instance I_k, k = 1, ..., i, contains input values v_{k1}, v_{k2}, ..., v_{kp} for the input registers r_1, r_2, ..., r_p, respectively, and output values u_{k1}, u_{k2}, ..., u_{kq} for the output registers t_1, t_2, ..., t_q, respectively. When the region is encountered again, the current values in registers r_1, r_2, ..., r_p may match one of the instances, say I_1. In this case, the output values u_{11}, u_{12}, ..., u_{1q} can be copied to the output registers t_1, t_2, ..., t_q, respectively and the computation can proceed with the code after the computation region immediately.

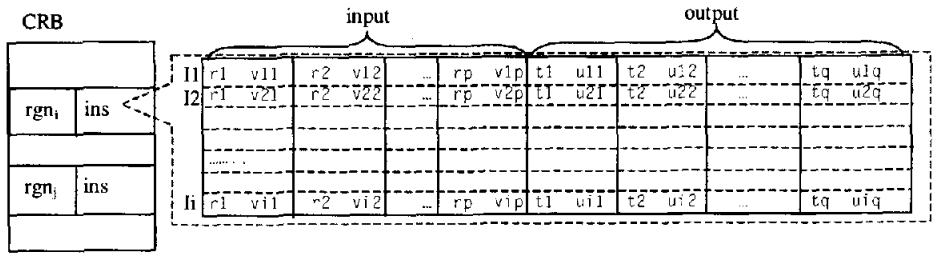


Figure 1. Hardware structure of computation reuse buffer

The compiler forms the computation regions. The compiler first identifies regions of code that exhibit high reusability. The starting point of each region is designated by a special instruction "reuse". Each computation region has a main exit and may have a few side exits. The main exit is identified by a "reuse.exit" instruction and the side exits are identified by the "reuse.abort" instructions. When a region is executed starting at the "reuse" instruction and ending at the main exit (reuse.exit), the input and output of the region is remembered in an instance for the region. If the execution reaches a side exit (reuse.abort), no instance will be remembered. The compiler also makes sure that each region has no more live-in or live-out registers than the hardware can memorize.

For the scheme proposed in [5], the compiler needs to insert an invalidation instruction after each store instruction that may access the same address as a memory operation (either a load or a store) inside a computation region. When an invalidation instruction is executed, the hardware marks the CRB entry for computation region specified in the invalidation instruction as invalid. An invalid region will not be reused. However, the invalidation instructions often have to be inserted conservatively as alias analysis is a known hard problem. In addition, even when a store accesses the same memory location as that by a memory operation in a computation region, the stored value may remain unchanged and we may still reuse the region. As shown in [14], majority of the invalidation operations by stores may be unnecessary.

To prevent the unnecessary invalidation, we introduce the concept of *speculative reuse* that takes advantage of the speculative multithreading hardware described in this paper. In speculative reuse, there is no need to insert invalidation instructions. Instead, the output in the matching instance can be "predicted" to be correct and is speculatively reused, even though some store executed outside the region may have accessed the same address as a memory operation inside the region. A computation region with potentially aliased memory operations is called an *aliased region*, and is identified with a special version of the "reuse" instruction, named "reuse.speculative". The speculative multithreading hardware is used to overlap the speculative execution of the code after the region with the execution of the computation region that verifies the predicted output. The speculative reuse will not commit result to memory or register file until the verification thread confirms that the predicted output is correct. If the verification thread finds out that the reused output is incorrect, the speculative execution will be squashed. At that time, the output of the computation region is available from the verification thread and can be used in the code after the region non-speculatively.

3. Value prediction and integrated hardware

We can extend the speculative reuse further to a more general integration of computation reuse and value prediction. In the integrated approach, if no instance matches the current input at the start of a region, we can use the instances recorded for the region as the history or context to predict the output of the region. We use the Reuse and Value Prediction Buffer (RVPB) shown in Figure 2 to maintain instances and to predict output values.

The information stored in the instances can be used to predict the output values using a context-based predictor, a stride predictor, a last value predictor, or a hybrid of them. A context-based predictor uses the recent history of previous output values to index a value table to predict a new output value. A stride predictor uses the latest output value and the difference between the latest two output values to predict a new output value. A last value predictor uses the most recent output as the predicted output.

To obtain the recent history for value prediction, we maintain a queue per region that contains pointers to the instances, with the most recently accessed instance first. The queue may contain repeated entries pointing to the same instance. We call the queue the *temporal queue (TQ)*. The hardware maintains the *temporal queue* as follows. Every time an instance is created or reused, it is added to the beginning of the queue. When an instance is replaced, it is removed from the queue.

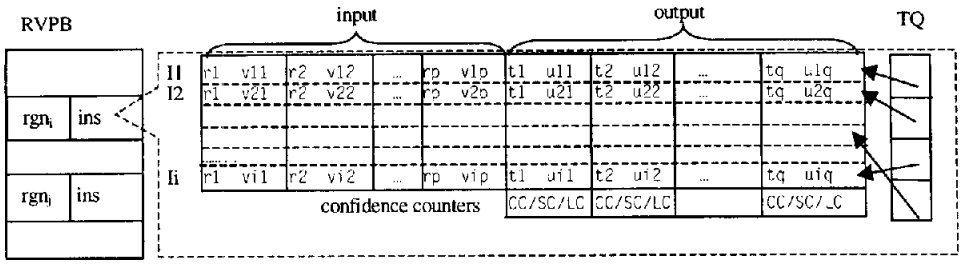


Figure 2. Hardware structure of reuse and value prediction buffer

Our value prediction is similar to the hybrid predictor proposed in [30]. For each output register in an instance, we use three 2-bit confidence counters (CC for context based predictor, SC for stride predictor, and LC for last value predictor) to indicate the prediction confidence of the predictors. A confidence counter is incremented each time the predicted value is correct and decremented when the value is mis-predicted. For an instance with eight live-in registers (7-bits each) and their values (64-bits each), and eight live-out registers and their values, the confidence counters use less than 0.5% of additional buffer space.

Continue with the example for region rgn_i in Figure 2. Assume we want to predict the output values $x_1, x_2, \ldots, x_q$ in the output registers $t_1, t_2, \ldots, t_q$, respectively. We predict each of the output values individually. Without loss of generality, we show how to predict the value x_1 in output register t_1. For simplicity, we assume that the top instances pointed to by TQ are I_1, I_2, and I_k. We first compare the confidence counters to select a predictor. If the last value predictor has the highest confidence, we use the value u_{11} as the predicted value for x_1. If the stride predictor has the highest confidence value, we use a stride of $(u_{11} - u_{21})$ to predict the value, namely $x_1 = u_{11} + (u_{11} - u_{21})$. If the context-based predictor has the highest confidence value, we hash the history values u_{11}, $u_{21}, \ldots, u_{k1}$, together with the identity of the computation region, such as IP of the reuse instruction and output register number, to obtain an index to a value prediction table (VPT, not shown in Figure 2) and use the value in the VPT entry as the predicted value. Namely, x_1 = VPT (hash $(u_{11}, u_{21}, \ldots, u_{k1}$, IP, $t_1)$).

For efficiency, a value predictor may store pre-computed values, such as the stride, partial context for each output register.

4. Architectural extension

The integrated approach requires the following architectural extension.

- A *reuse* instruction. This instruction marks the beginning of a non-aliased computation region.
- A *reuse.speculative* instruction. This instruction marks the beginning of an aliased computation region.
- A *reuse.exit* branch instruction. This instruction marks the main exit of a computation region.
- A *reuse.abort* branch instruction. This instruction marks a side exit of a computation region.
- A live out hint on an instruction that defines a register inside a computation region. This hint indicates that the value of the defined register is live-out of the computation region.

5. Speculative multithreading processor

We propose a speculative multithreading processor to overlap the executions of the computation region with the speculative execution of the code after the region. The proposed processor has two cores, a *Main core* (*M core*) and a *Check core* (*C core*).

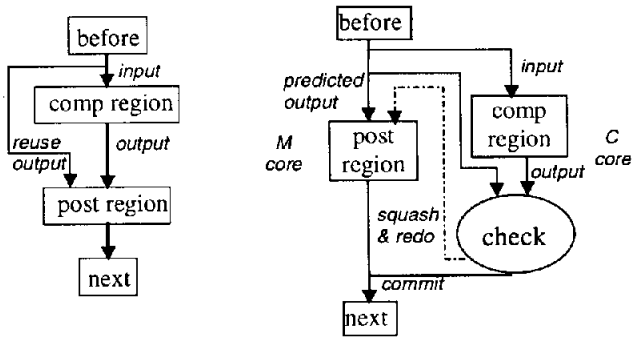


Figure 3. Sequential and multi-threaded execution

Figure 3 (a) shows the sequential execution of a computation region and the code after the region (post region) on a processor with computation reuse support. The computation region can be non-speculatively skipped only when the output can be reused. Figure 3 (b) shows the speculative execution on our multi-threaded processor. The code in post region is executed on the M core with the predicted output for the computation region. The code in the computation region is executed in

the C core. The C core also compares its output against the predicted output. If the outputs are the same, the speculatively executed code (post region) in M core will be committed. Otherwise the speculative execution is squashed and repeated. Notice that when the M core executes speculatively the code after computation region (post region), it may encounter another computation region. The multi-threaded processor needs to handle cascaded computation regions efficiently.

To handle cascaded computation regions, we propose a multi-threaded microarchitecture as shown in Figure 4. The M core has a "persistent register file" (P register file), which is used during non-speculative execution. It also has a shadow register file (S register file) that is used only during speculative execution. It has a number of write-back buffers for results (memory and register) generated in speculative mode. The C core is similar to a traditional processor. It has a local register file (L register file). The three register files in M and C cores all have the same structure.

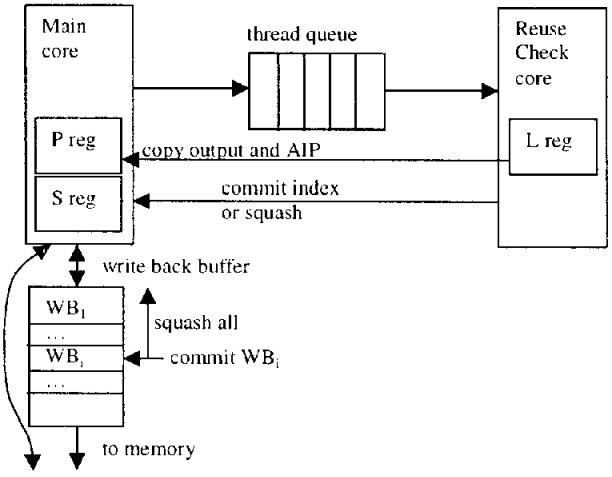


Figure 4. Multithreaded microarchitecture

There is a *thread queue* between the two cores for communicating thread records. Each thread record represents the execution of a computation region. The thread record contains the starting IP of the computation region (SIP), the input registers and their values, and the output registers and their speculated values. It also contains the IP for the instruction after the main exit of the computation region (AIP) and the index of the WB buffer used by the speculatively executed code after the main exit of the computation region. Each WB buffer contains a set of register values and memory updates. The size of a WB is implementation dependent.

Each program starts execution in non-speculative mode. During non-speculative execution, the C core is idle, and the M core uses the P register file and updates memory directly. It remains in non-speculative mode when a computation region is skipped due to non-speculative reuse of its output values. The execution of the M core enters speculative mode when one of the following *speculation conditions* is encountered.

- A "reuse" instruction is executed, and no matching instance is found but the output can be predicted, or
- A "reuse.speculative" instruction is executed, and either a matching instance is found or the output can be predicted.

When a speculation condition is encountered, the M core uses the RVPB to determine the output to be speculatively used by the code after the main exit of the computation region. Then, it creates a new thread record for the code in the computation region and enters the record in the thread queue. When M code switches to speculative mode, the S register file is identical to the P register file. During speculative execution, the M core accesses and updates the S register file. It also sends output register updates and memory updates to the WB buffer (the M core may need to stall when the WB buffer is full).

During speculative execution, the M core may encounter another *speculation condition* that will cause a new thread record to be created and entered in the thread queue (or stalls when the thread queue is full). The speculative execution between two consecutive speculation conditions uses a WB buffer. In other words, when the M core creates a new thread while in speculative mode, it marks the end of the current WB buffer, e.g. WB_{i-1}, and the speculative execution starts to use the next WB buffer, e.g. WB_i. The WB buffers can be committed individually.

The C core repeatedly fetches a thread record from the thread queue. It copies the input register values in the thread record into the local register file and starts execution from the thread's starting IP address (SIP). When the reuse.exit instruction is reached, it compares the output register values with the predicted output values in the thread record. If the values match, it sends a "commit index" request to the M core, where index is the index of the WB buffer stored in the thread record. If C core determines that its output values are different from predicted output values, it sends a "squash" request to the M core to squash speculative execution. The C core also updates the corresponding instance with the new output values when the predicted values are wrong.

When the M core received the "commit index" request, it commits all the results in the WB buffer indexed by "index" to memory and the P register file. The committed WB buffer will be released and made available for later uses. The M core will switch to non-speculative mode if the committed WB buffer is the last WB buffer being used. When the M core received the

"squash" signal, it first squashes all the WB buffers. Then it copies the output values from C core to P register files and resumes execution at the instruction pointed to by AIP. The M core will then execute in non-speculative mode.

M core and C core need to synchronize memory accesses. When conflict occurs, M core will be squashed.

6. Experimentation

We evaluate the integrated reuse and value prediction technique with an IA-64 [12] research compiler on the SpecInt95 benchmarks, with the "lite" inputs used internally for performance study (the "lite" inputs exercise about 300M instructions per benchmark).

In this Section, we describe our experimentation for the integrated approach. We first describe the region formation algorithm. After that we describe our performance measurement methodology and the experimental configuration. Next we show the performance improvement of the integrated approach over the computation reuse and value prediction alone. After that, we examine the factors that affect the performance. Then, we show that the region size is an important indication for performance, especially when the reuse instruction takes multi-cycle overhead. We identify the limiting parameters and propose methods for increasing region sizes. Finally, we measure the sensitivity of the approaches to the value profile used to form computation regions. One interesting result is that the integrated approach is insensitive to the value profile. In fact we can apply the technique without using value profile and still obtain outstanding results

6.1 Compiler region formation

To form computation regions, the compiler first profiles the programs to collect value profile [3]. Instructions, such as entry, call, ret, and indirect branch, are not candidates for computation region and are not profiled. For each candidate instruction or innermost loop, the value profiler collects the "top-value probability", TVP, the probability that one of the top-5 sets of input values will occur at any execution of the instruction or the loop. Then, it groups candidate instructions into multi-entry multi-exit regions with the following constraints.

- An instruction or innermost loop included inside a region must have a TVP value no less than a threshold (e.g. 30%).
- An instruction included inside a region must be frequently executed (e.g. executed at least 50 times).
- Each region must contain a minimum number (e.g. 5) of instructions or contains an innermost loop.
- The probability of execution from main entry to main exit is high (e.g. at least 70%).
- The prolog block of any side entry is either rarely executed (e.g. block frequency < 10) or it branches to the side entry block with a low probability (e.g. less than 10%).
- The number of live-in and live-out registers is no more than the hardware threshold (e.g. 8 live-ins and 8 live-outs).
- We currently only allow integer and floating point registers to be live-in and live-out of a region. In the future, we will relax this constraint to allow predicate and control registers.

In addition to value profile information, the compiler also employs edge profile information, tail duplication and instruction reordering during region formation.

6.2 Performance measurement methodology

We use compiler-driven simulation to evaluate the integrated approach. In the compiler-driven simulation, entries and exits of each computation region are instrumented to make calls to a library that simulates the RVPB hardware. We simulate a hybrid predictor with a 2-delta stride predictor and a last value predictor in RVPB (the context-based predictor is not implemented yet). The simulation collects for each region the probability that the region can have its output values reused or correctly predicted. We call the probability the ***success probability*** for the region. After the simulation, the compiler uses the frequency and cycle count for each basic block B, $F(B)$ and $C(B)$, and the frequency and success probability for each region R, $F(R)$ and $SP(R)$, to calculate the dynamic cycle counts with and without reuse and value prediction. Notice that IA-64 is a statically scheduled processor and we can compute execution cycle for each basic block after instruction scheduling. The compiler-computed cycle counts assume ideal cache and perfect branch prediction.

The speedup is measured by T_1/T_2, where T_1 is the dynamic cycle count without reuse or value prediction, and T_2 is the dynamic cycle counts with reuse and value prediction. T_1 and T_2 can be specified in the following formulas, where *oh* is the overhead for the reuse instruction. We subtract one from the *oh* in the formula for T_2 because the reuse instruction has already been charged one cycle in C(B).

$$T_1 = \sum_{\forall \text{ basic block } B} F(B) * C(B)$$

$$T_2 = \sum_{\forall \text{ region } R} \left[F(R) * (oh - 1) + (1 - SP(R)) * \sum_{\forall B \in R} \left(F(B) * C(B) \right) \right] + \sum_{\forall B \in \text{ any region}} F(B) * C(B)$$

6.3 Experimental configuration

All experiment is based on the following ***base configuration*** with adjustment for the needs of specific experiment: the RVPB hardware has 128 entries. Each entry has four instances. Each instance has eight input registers and their values, and eight output registers and their values. The base RVPB hardware costs about 64K bytes of buffer space. The compiler forms regions by grouping instructions or innermost loops with TVP greater or equal to 30% into computation regions. The instruction latencies and the processor pipeline are the same as the Itanium processor.

6.4 Performance measurement

The speedup result with the base configuration and one-cycle overhead for the reuse instruction is shown in Figure 5. Each benchmark has four bars. The first bar (reuse4) is for the speculative computation reuse with four instances per region. The second bar (reuse8) is for the speculative computation reuse with eight instances per region. This bar represents the upper bound for the non-speculative computation reuse as proposed in [5]. The third bar (reuse4+vp) is for the integrated approach with reuse4 and the hybrid stride and last value predictor. The forth bar (vp) is for a hybrid stride and last value predictor alone.

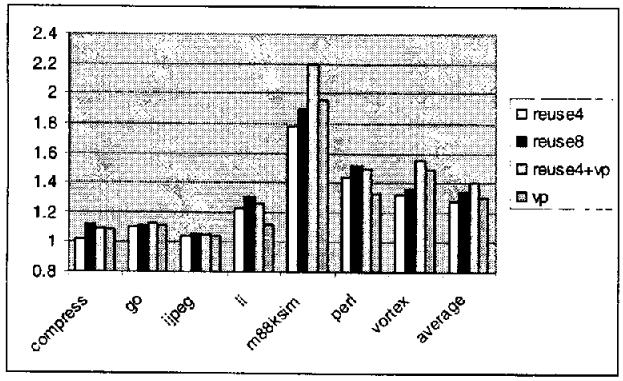


Figure 5. Speedups with one cycle overhead

The integrated approach out-performs either reuse4 (from a speedup of 1.25 for reuse to that of 1.40 for the integrated approach) or value prediction (from 1.28 to 1.40). In particular, the integrated approach uses only four instances per region, and still it out-performs computation reuse with eight instances per region (from 1.33 to 1.40).

The speedup with two-cycle overhead is shown in Figure 6. The higher overhead reduces the overall speedups, but the integrated approach shows similar scale of improvement over computation reuse or value prediction alone.

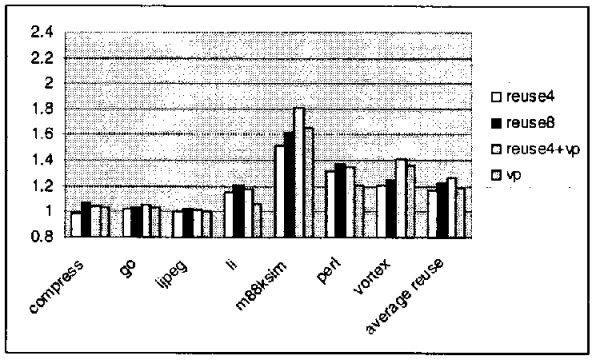


Figure 6. Speedups with two cycle overhead

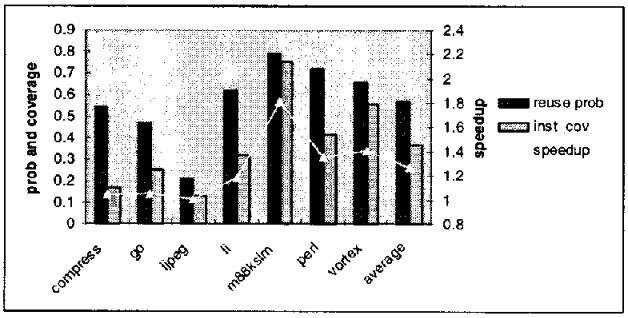


Figure 7. Success probability, coverage, and speedup

6.5 Success probability and coverage

Several benchmarks, such as compress, go, and ijpeg, show only slight performance improvement while some other benchmarks, such as m88ksim, perl, and vortex, show significant performance improvement. To understand this difference, we notice that the speedup of the integrated approach is affected by the following factors:

Success probability: the probability that the output of a computation region can be reused or correctly predicted.

Coverage: the percentage of dynamic code that is included inside computation regions.

The two factors work together and affect the performance. Figure 7 shows the relationship between the success probability, coverage, and the speedup for the benchmarks. The data are collected using the base configuration with 2-cycle overhead. The three worst benchmarks (compress, go, and ijpeg) all have low

coverage. For example, ijpeg has only 13% code inside computation regions. They also have lower success probabilities than the other benchmarks. For example, computation regions in ijpeg are reused or correctly predicted in only 25% of the times.

We may increase the success probability of the regions by only allow highly reusable instructions in the regions. However, this often makes regions smaller and reduces the coverage. We may also try to increase the coverage by including instructions that are less likely to be reusable inside the regions at the risk that the regions will have lower success probability. It is an on-going effort to identify region formation heuristics that increase coverage without decreasing the success probability.

6.6 Region size

From Figure 5 and Figure 6, we can see that when the overhead of reuse instruction increases from one cycle to two cycles, the average speedup drops significantly from 1.40 to 1.27. The drop can be attributed to the relatively small size of the computation regions. For the base configuration, the region size is about 15 instructions on the average. This represents a few cycles per region for the six issue IA-64 processors. The one cycle additional overhead to each region eats into the performance benefit quickly.

We use the number of static instructions to measure the size of a region. The average size of the regions is weighted by the execution frequencies of the regions. This measurement of region size could be conservative as our regions can contain loops and a loop may iterate multiple times inside a region.

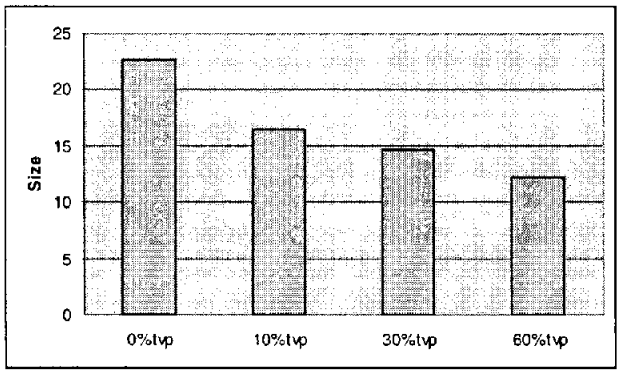


Figure 8. Region size v.s. TVP thresholds

Region size is constrained by two configuration parameters: the TVP threshold that qualifies instructions for including inside a region, and the maximum number of live-ins and live-outs that a region can have. Therefore, one way to increase the region size is to reduce the TVP threshold. Figure 8 shows the average size when the TVP thresholds change. As the TVP threshold is relaxed from 60% to 0%, the region size increases from 13 to 23 instructions. The 0% threshold means no value profile is used. Later we will see that with 0% TVP threshold our integrated approach still delivers good performance.

To further increase the region size, we need to relax the constraint on the maximum number of live-ins and live-outs for a region. Figure 9 shows the average region size when the maximum number of live-in and live-out are increased (with the base configuration but changing the number of live-in and live-out registers and using the 0% TVP threshold). Clearly, the number of live-in and live-outs imposes severe limitation to the size of the computation regions. In other words, if we allow more live-in/outs to a region, the region size will increase significantly. For example, if we allow computation regions to increase the number of allowable live-in/out values from 8 to 32, the average region size increases from 23 to 62 instructions. However, increasing the number of live-ins and live-outs directly increases the hardware requirement. For example, to allow 32 live-ins and 32 live-outs, each instance needs to remember up to 32 input values and 32 output values.

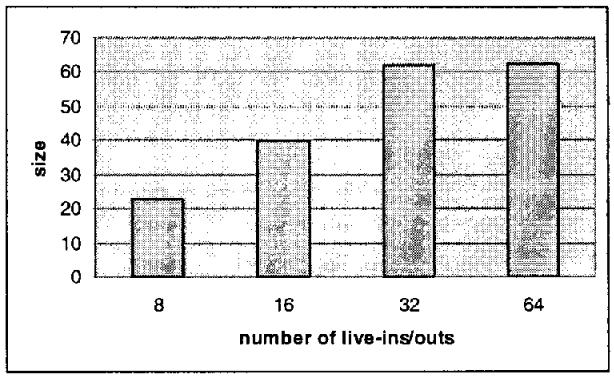


Figure 9. Region size v.s. live-in/out constraints

We are currently working on a compiler transformation that relaxes the live-in/out restriction. In general, the compiler can rearrange code to minimize the lifetime of the register values. This will help to enlarge regions under a given live-in and live-out constraint.

We are also investigating enhancement to the integrated framework so not all live-in and live-out values have to be available at the region entries. This may allow large computation regions to be reused without increasing hardware cost.

6.7 Sensitivity to value profile

One thing worth noticing is that the integrated approach is less sensitive to the TVP used to form

computation regions than computation reuse does. Figure 10 shows result with the base configuration, two cycles overhead, and changing TVP thresholds. When the threshold of TVP is changed the performance of either reuse or value prediction varies significantly, while the performance of the integrated approach remain relatively stable. For example, when the TVP changes from 0% to 60%, the speedup for reuse4 increases from 1.11 to 1.22. That is because that a computation region consisting of instructions with high TVP generally has smaller numbers of reuse instances and thus more reusable. Our experience indicates that value profile is quite expensive to collect. This data suggests that computation reuse may not be practical without additional runtime management hardware [6]. In contrast, when the threshold of TVP changes from 0% to 60%, the speedup for value prediction decreases from 1.27 to 1.16. In other words, the performance of value prediction decreases as the TVP threshold increases. This is partially because that we currently only use stride predictor and last value predictor. Both of the predictors do not require instructions with high TVP to predict the region output accurately. Decreasing the threshold for TVP, nevertheless, allows more instructions to be included in computation regions. This increases the overall performance for value prediction.

For the integrated approach, the performance improvement is a combination of those for both computation reuse and value prediction, with value prediction affects the combined performance more than the computation reuse does. The best configuration for the interpreted approach is with 0% TVP threshold. This indicates that the performance of our integrated approach does not depend on accurate value profiles and thus enables it to be used in production systems.

7. Conclusions and future work

In this paper, we describe integrated approach that combines the strengths of both computation reuse and value prediction in an integrated framework. For the SpecInt95 benchmarks, our experiment shows that the integrated approach significantly out-performs either computation reuse or value prediction alone. For example, with one cycle overhead, the integrated approach improves over computation reuse from a speedup of 1.25 to 1.40, and improves over value prediction from 1.28 to 1.40. In particular, the integrated approach out-performs a computation reuse configuration that has twice as much reuse buffer entries (from a speedup 1.33 to 1.40), although need an additional hardware core to support speculation. Furthermore, unlike the computation reuse approach [5], the performance of the integrated approach does not rely on value profile during region formation and thus our approach is more suitable for production systems.

In the future, we will continue the work in the following areas:

- Evaluate our integrated approach on a cycle-accurate simulator for performance and hardware overhead.
- Investigate the possibility of using a general speculative multithreaded processor (e.g. SMT) to support the integrated reuse and value prediction.
- Evaluate the effects of adding a context-based value predictor into the integrated approach.
- Fine-tune the heuristic for region formation to increase coverage as well as the success probability.
- Implement the compiler algorithm to minimize live-in/live-out to computation regions.
- Design enhancement to the integrated framework so not all live-in and live-out values have to be available at the region entries. This may enable larger computation regions to be formed without increasing hardware cost.
- Investigate other forms of value profile from TVP to further improve the performance of the integrated approach.

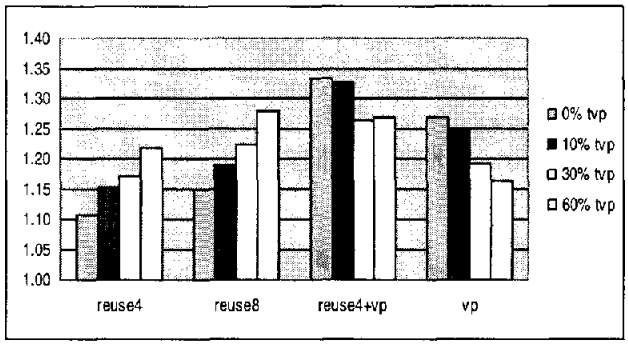


Figure 10. Average speedups with two-cycle overhead vs. TVP thresholds

8. Acknowledgements

We would like to thank John Shen, Guri Sohi, and Mauricio Serrrano for valuable comments and suggestions. Daniel Connors helped us to understand several implementation issues with compiler-directed computation reuse. Brad Calder provided helpful insights into the value-profiling algorithm [3]. Our colleagues at the MRL compiler research labs provided constant support to this work. We appreciate the comments from the anonymous reviewers that help improve the quality of the paper.

References

1 H. Akkary and M. Driscoll. A Dynamic Multithreading Processor. *Proc. of the 31st Annual International Symp. on Microarchitecture*, pp.226 -236, Dec. 1998.

2 T. Austin. DIVA: A Reliable Substrate for Deep Submicron Microarchitecture Design. *Proc. of the 32nd Annual International Symp. on Microarchitecture*, pp.196-207, Nov. 1999.

3 B. Calder, P. Feller, and A. Eustance. Value Profiling. *Proc. of the 30th Annual International Symp. on Microarchitecture*, Dec. 1997.

4 S. Chatterjee, C. Weaver, and T. Austin. Efficient Checker Processor Design. *Proc. of the 33rd Annual International Symp. on Microarchitecture*, Dec. 2000.

5 D. Connors and W.-M. Hwu. Compiler-Directed Dynamic Computation Reuse: Rationale and Initial Results. *Proc. of the 32nd Annual International Symp. on Microarchitecture*, Nov. 1999.

6 D. Connors, H. Hunter, B.-C. Cheng, and W.-M. Hwu. Hardware Support for Dynamic Activation of Compiler-Directed Computation Reuse. *Proc. of the 9th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp.222-233, Nov. 2000.

7 S. Dutta and M. Franklin. Block-Level Prediction for Wide-Issue Superscalar Processors. *Proc. of the 1st International Conference on Algorithms and Architectures for Parallel Processing*, Vol. 1, pp.143-152, 1995.

8 C. Fu, M. Jennings, S. Larin, and T. Conte. Value Speculation Scheduling for High Performance Processors. *Proc. of the 8th International Conference on Architectural Support for Programming Languages and Operating Systems*, Oct. 1998.

9 F. Gabbay and A. Mendelson. Using Value Prediction to Increase the Power of Speculative Execution Hardware. *ACM Trans. On Computer Systems*, Vol. 16, No. 3, pp.234-270, Aug. 1998.

10 Jian Huang and D.J Lilja, Extending value reuse to basic blocks with compiler support. *IEEE Trans. on Computers*, Vol. 49 No. 4, pp. 331-347. April 2000

11 Jian Huang and D.J Lilja, Exploring sub-block value reuse for superscalar processors. *Proc. International Conference on Parallel Architectures and Compilation Techniques*, pp. 100 -107. 2000

12 Intel Corp. *Intel IA-64 Architecture Software Developer's Manual*, Jan. 2000. Vol. 1-4. http://developer.intel.com/design/ia-64/manuals/index.htm.

13 V. Krishnan, J. Torrellas. A Chip-Multiprocessor Architecture with Speculative Multithreading. *IEEE Transactions on Computers*, Vol.48, No.9, pp.866-880, Sept. 1999.

14 K. Lepak and M. Lipasti. On the Value Locality of Store Instructions. *Proc. of the 27th Annual International Symp. on Computer Architecture*, pp. 182-191, June 2000.

15 M. Lipasti, C. Wilkerson, and J. Shen. Value Locality and Load Value Prediction. *Proc. of the 7th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp.138-147, Oct. 1996.

16 M. Lipasti and J. Shen. Exceeding the Dataflow Limit via Value Prediction. *Proc. of the 29th Annual International Symp. on Microarchitecture*, pp.226-237, Dec.1996.

17 Chi-Keung Luk. Tolerating Memory Latency through Software-Controlled Pre-Execution in Simultaneous Multithreading Processors, *Proc. of the 28th Annual International Symp. on Computer Architecture*, July 2001.

18 P. Marcuello, J. Tubella, and A. Gonzalez. Value Prediction for Speculative Multithreaded Architectures. *Proc. of the 32nd Annual International Symp. on Microarchitecture*, pp.230-236, Nov. 1999.

19 T. Nakra, R. Gupta, and M. Soffa. Value Prediction in VLIW Machines. *Proc. of the 26th Annual International Symp. on Computer Architecture*, pp.258-269, May 1999.

20 Z. Purser, K. Sundaramoorthy, and E. Rotenberg. A Study of Slipstream Processors. *Proc. of the 33rd Annual International Symp. on Microarchitecture*, pp.269-280, Dec. 2000.

21 Y. Sazeides and J. Smith. The Predictability of Data Values. *Proc. of the 30th Annual International Symp. on Microarchitecture*, pp.248-258, Dec. 1997.

22 M. J. Serrano, *Performance Tradeoffs in Multistreamed Superscalar Architectures*, Ph.D. Dissertation, University of California, Santa Barbara, 1994.

23 A. Sodani and G. Sohi. Dynamic Instruction Reuse. *Proc. of the 24th Annual International Symp. on Computer Architecture*, pp.194-205, July 1997.

24 A. Sodani and G. Sohi. An Empirical Analysis of Instruction Repetition. *Proc. of the 8th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp.35-45, Oct. 1998.

25 A. Sodani and G. Sohi. Understanding the Difference between Value Prediction and Instruction Reuse. *Proc. of the 31st Annual International Symp. on Microarchitecture*, pp.205 -215, Dec. 1998.

26 G. Sohi, S. Breach, and T. Vijaykumar. Multiscalar Processors. *Proc. of the 22nd Annual International Symp. on Computer Architecture*, pp.258-269, pp.414-424, June 1995.

27 K. Sundaramoorthy, Z. Purser, and E. Rotenberg. Slipstream Processors: Improving both Performance and Fault Tolerance. *Proc. of the 9th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp.257-268, Nov. 2000.

28 D. Tullsen, S. J. Eggers, and H.M. Levy. Simultaneous Multithreading: Maximizing on-chip Parallelism. *Proc. of the 22nd Annual International Symp. on Computer Architecture*, pp.392-403, June. 1995.

29 D. Tullsen and J. Shen. Storageless Value Prediction Using Prior Register Values. *Proc. of the 26th Annual International Symp. on Computer Architecture*, May 1999.

30 K. Wang and M. Franklin. Highly Accurate Data Value Prediction using Hybrid Predictors. *Proc. of the 30th Annual International Symp. on Microarchitecture*, pp.281-290, Dec. 1997.